Algorithmic Optimization and Implementation of an Incompatibility-Based Classifier

Alexandre Lebleu, Léonard Jung
Supervisor: Chunyang Fan
Sorbonne Université

May 2025

Contents

1 Introduction **1**
1.1 Context . . . 1
1.2 Motivation and objectives of the project . . . 1
1.3 Organization of the report . . . 1

2 Theoretical foundations **1**
2.1 Situations, case base and similarity measures . . . 1
2.2 Incompatibility between triplets . . . 2
2.3 Prediction by minimizing incompatibility . . . 2
2.4 Naive algorithm . . . 2

3 Algorithmic improvement **3**
3.1 Mathematical analysis . . . 3
3.2 Proposed optimized algorithm . . . 7

4 Improvement through parallelization **8**
4.1 Introduction to PyTorch . . . 8
4.2 From loop-based to parallel computation . . . 9

5 Experiments **9**
5.1 Implementation . . . 10
5.1.1 Functions shared by the native Python versions . . . 10
5.1.2 Naive Python version . . . 11
5.1.3 Optimized Python version . . . 11
5.1.4 PyTorch version . . . 11
5.2 Results and experiments . . . 12
5.2.1 Illustration of the prediction . . . 12
5.2.2 Comparison of the Python versions . . . 12
5.2.3 PyTorch version . . . 13

6 Limitations and possible improvements **15**
6.1 Limitations of parallel computing with PyTorch . . . 15
6.2 Possible improvements . . . 16

7 Conclusion **16**

1 Introduction

1.1 Context

Complexity-based Analogical Transfer (CoAT) [1] is a case-based prediction (CBP) method [2] that belongs to the field of machine learning. It aims to predict the outcome of a new case by relying on similar cases encountered previously. CoAT is based on analogical transfer [3], a cognitive mechanism according to which a solution to a new problem can be built by drawing on a similar problem already encountered. In artificial intelligence, this principle is exploited by comparing past situations and transferring the solution of a source case to a target case, provided that the two are sufficiently similar. This transfer assumes consistency between the similarity of the situations (inputs) and that of their solutions (outputs). If two cases are close in the input space, then their outputs must be close as well; otherwise, analogical reasoning fails.

1.2 Motivation and objectives of the project

The algorithmic complexity of the CoAT classifier is an obstacle to its application at scale. Indeed, computing a prediction involves a large number of comparisons, which leads to a very high computation time. In particular, the author uses a **triple** "for" loop to perform the computations, without any parallel optimization, which leaves room for improving the algorithm. It is this twofold motivation, optimizing the implementation of CoAT and analyzing its behavior on a concrete case, that justifies the present project. It aims at:

- a thorough understanding of how CoAT and the incompatibility measure work;
- implementing a naive version in Python for validation;
- analyzing its complexity and developing an optimized version;
- moving to a parallel version with PyTorch to take advantage of vectorized computation;
- an experimental comparison on simulated or real datasets.

1.3 Organization of the report

In the next section, we detail the mathematical foundations of the CoAT method and the formal definition of incompatibility. We then present the naive algorithm and our first optimized version. The fourth section is devoted to the parallel implementation in PyTorch. Finally, we analyze the performance obtained, discuss the limitations encountered, and outline possible improvements.

2 Theoretical foundations

In this section, we introduce the notation and mathematical definitions that will be used to formalize how the CoAT classifier works.

2.1 Situations, case base and similarity measures

We denote:

- $\mathcal{X}$: the input space, that is, the set of possible representations of situations;
- $\mathcal{Y}$: the output space, representing the labels or outcomes associated with the inputs.
- A situation is defined as a pair:

$$z_i = (x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$$

where x_i is an input and y_i is the associated output.

- The set of cases forms the case base, denoted:

$$\mathcal{CB} = \{z_1, z_2, \ldots, z_n\}$$

  with n the number of observed cases.

To compare cases with one another, we introduce two similarity functions:

- $\sigma_X : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, a similarity measure between inputs;
- $\sigma_Y : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$, a similarity measure between outputs.

These functions assign a real value indicating how similar two elements are. The higher the value, the stronger the similarity; they are maximal when the elements are identical.

2.2 Incompatibility between triplets

The CoAT method relies on the principle that similarity in the input space must be consistent with similarity in the output space.

A triplet $(z_i, z_j, z_k) \in \mathcal{CB}^3$ is said to be incompatible if:

$$\sigma_X(x_i, x_j) \geq \sigma_X(x_i, x_k) \quad \text{and} \quad \sigma_Y(y_i, y_j) < \sigma_Y(y_i, y_k) \tag{1}$$

In other words, x_j is closer to (or as close to) x_i than x_k in the input space, but y_j is less close to y_i than y_k in the output space. This is called a similarity inversion.

We define the incompatibility function Γ as the total number of incompatible triplets in the case base:

$$\Gamma(\sigma_X, \sigma_Y, \mathcal{CB}) = \left|\left\{(z_i, z_j, z_k) \in \mathcal{CB}^3 \;\middle|\; \sigma_X(x_i, x_j) \geq \sigma_X(x_i, x_k) \wedge \sigma_Y(y_i, y_j) < \sigma_Y(y_i, y_k)\right\}\right|$$

This function measures the overall consistency of the case base with respect to the principle of analogical transfer.

This formulation allows a direct and efficient implementation of the count of inconsistencies in the case base.

2.3 Prediction by minimizing incompatibility

Given a new input $x_{\text{new}} \in \mathcal{X}$, CoAT predicts the output $\hat{y}_{\text{new}}$ by minimizing the incompatibility resulting from adding the case (x_{new}, y) to the case base:

$$\text{CoAT}(x_{\text{new}}) = \hat{y}_{\text{new}} = \arg\min_{y \in \mathcal{Y}} \Gamma\left(\sigma_X, \sigma_Y, \mathcal{CB} \cup \{(x_{\text{new}}, y)\}\right)$$

This strategy selects the output that is most consistent with the structure of the cases already observed.

2.4 Naive algorithm

The naive $\mathcal{O}(n^3)$ algorithm for the CoAT classifier proposed in [1] consists in going through all possible triplets (z_i, z_j, z_k) in the case base together with the new situation $z_{new} = (x_{new}, y_{new})$. For each triplet, we check whether constraint 1 is violated.

Algorithm 1: Naive detection of similarity inversions

Data: A set $E = \{(x_i, y_i)\}_{i=1}^{n}$
Result: Total number of similarity inversions

```
inversions ← 0
for i ← 1 to n do
    for j ← 1 to n do
        for k ← 1 to n do
            if σ_X(x_i, x_j) ≥ σ_X(x_i, x_k) and σ_Y(y_i, y_j) < σ_Y(y_i, y_k) then
                inversions ← inversions + 1
return inversions
```

3 Algorithmic improvement

In this section, we detail the work that allowed us to improve the complexity of the CoAT classifier.

Based on the analysis presented below, we observe that triplets that do not contain the new situation $z_{new} = (x_{new}, y_{new})$ are of no use for the prediction. Therefore, to improve efficiency, this optimized version only tests the triplets in which the new situation z_{new} appears, which reduces the complexity to $\mathcal{O}(n^2)$.

More precisely, we insert z_{new} in each of the possible roles in a triplet (i, j, k), and loop over all remaining pairs (i, j) of the case base. This makes it possible to detect all similarity inversions caused by z_{new}, without recomputing the relations internal to the existing case base.

3.1 Mathematical analysis

The first observation is that the function Γ can be rewritten as a sum of indicator functions, i.e.

$$\Gamma(\sigma_X, \sigma_Y, \mathcal{CB}) = \sum_{(z_i, z_j, z_k) \in \mathcal{CB}^3} \mathbb{I}_{\text{inc}}(z_i, z_j, z_k)$$

where

$$\mathbb{I}_{\text{inc}}(z_i, z_j, z_k) = \begin{cases} 1 & \text{if equation 1 is satisfied} \\ 0 & \text{otherwise} \end{cases}$$

Lemma 1. *Let z_{new} be a new situation. The set of triplets on the augmented case base $(CB \cup \{z_{new}\})^3$ can be decomposed as follows:*

$$(\mathcal{CB} \cup \{z_{new}\})^3 = \mathcal{CB}^3 \sqcup U$$

where U is the set of triplets in which z_{new} appears at least once.

Proof. Since $\mathcal{CB}$ and $\{z_{new}\}$ are disjoint, the set of triplets decomposes into:

$$\begin{aligned}(\mathcal{CB} \cup \{z_{new}\})^3 = \mathcal{CB}^3 \\ \sqcup\ \{z_{new}\}^3 \sqcup\ (\mathcal{CB}^2 \times \{z_{new}\}) \sqcup\ (\mathcal{CB} \times \{z_{new}\} \times \mathcal{CB}) \sqcup\ (\{z_{new}\} \times \mathcal{CB}^2) \\ \sqcup\ (\mathcal{CB} \times \{z_{new}\}^2) \sqcup\ (\{z_{new}\} \times \mathcal{CB} \times \{z_{new}\}) \sqcup\ (\{z_{new}\}^2 \times \mathcal{CB})\end{aligned}$$

We then set:

$$U = \{z_{new}\}^3 \sqcup (\mathcal{CB}^2 \times \{z_{new}\}) \sqcup (\mathcal{CB} \times \{z_{new}\} \times \mathcal{CB}) \sqcup (\{z_{new}\} \times \mathcal{CB}^2) \\ \sqcup (\mathcal{CB} \times \{z_{new}\}^2) \sqcup (\{z_{new}\} \times \mathcal{CB} \times \{z_{new}\}) \sqcup (\{z_{new}\}^2 \times \mathcal{CB})$$

□

Proposition 1. *The set U of triplets containing the new situation z_{new} can be decomposed as:*

$$U = U_1 \sqcup U_2 \sqcup U_3 \sqcup U_4 \sqcup U_5 \sqcup U_6 \sqcup U_7$$

with:

$$\begin{aligned} U_1 &= \{(z_{new}, z_j, z_k) \mid (z_j, z_k) \in \mathcal{CB}^2\} \\ U_2 &= \{(z_i, z_{new}, z_k) \mid (z_i, z_k) \in \mathcal{CB}^2\} \\ U_3 &= \{(z_i, z_j, z_{new}) \mid (z_i, z_j) \in \mathcal{CB}^2\} \\ U_4 &= \{(z_{new}, z_{new}, z_k) \mid z_k \in \mathcal{CB}\} \\ U_5 &= \{(z_{new}, z_j, z_{new}) \mid z_j \in \mathcal{CB}\} \\ U_6 &= \{(z_i, z_{new}, z_{new}) \mid z_i \in \mathcal{CB}\} \\ U_7 &= \{(z_{new}, z_{new}, z_{new})\} \end{aligned}$$

This proposition follows directly from the decomposition of $(\mathcal{CB} \cup \{z_{new}\})^3$ obtained above.

Proposition 2. *Let $z_{new} = (x_{new}, y)$ be a new situation. The total incompatibility function satisfies:*

$$\Gamma(\sigma_X, \sigma_Y, \mathcal{CB} \cup \{z_{new}\}) = \underbrace{\Gamma(\sigma_X, \sigma_Y, \mathcal{CB})}_{\text{independent of } z_{new}} + \sum_{(z_i, z_j, z_k) \in U} \mathbb{I}_{inc}(z_i, z_j, z_k)$$

where $\mathbb{I}_{inc}$ is the indicator function of a similarity inversion.

Proof. By definition:

$$\Gamma(\sigma_X, \sigma_Y, \mathcal{CB} \cup \{z_{new}\}) = \sum_{(z_i, z_j, z_k) \in (CB \cup \{z_{new}\})^3} \mathbb{I}_{\text{inc}}(z_i, z_j, z_k)$$

From the decomposition of Lemma 1:

$$(\mathcal{CB} \cup \{z_{new}\})^3 = \mathcal{CB}^3 \sqcup U$$

Hence, $\Gamma(\sigma_X, \sigma_Y, \mathcal{CB} \cup \{z_{new}\})$ can be split into a sum:

$$\Gamma(\sigma_X, \sigma_Y, \mathcal{CB} \cup \{z_{new}\}) = \Gamma(\sigma_X, \sigma_Y, \mathcal{CB}) + \Gamma(\sigma_X, \sigma_Y, U)$$

We then have:

$$\Gamma(\sigma_X, \sigma_Y, \mathcal{CB} \cup \{z_{new}\}) = \Gamma(\sigma_X, \sigma_Y, \mathcal{CB}) + \sum_{(z_i, z_j, z_k) \in U} \mathbb{I}_{\text{inc}}(z_i, z_j, z_k)$$

□

Theorem 1. *Let x_{new} be a new input, and let $\mathcal{Y}$ be the set of possible values for its output. Then the CoAT algorithm predicts:*

$$\mathrm{CoAT}(x_{new}) = \arg\min_{y \in \mathcal{Y}} \Gamma(\sigma_X, \sigma_Y, \mathcal{CB} \cup \{z_{new}\})$$

and this expression is equivalent to:

$$\mathrm{CoAT}(x_{new}) = \arg\min_{y \in \mathcal{Y}} \sum_{(z_i, z_j, z_k) \in U(y)} \mathbb{I}_{inc}(z_i, z_j, z_k)$$

where $U(y)$ is the set of triplets containing the situation $z_{new} = (x_{new}, y)$.

Proof. Let $z_{new} = (x_{\text{new}}, y)$ be a candidate situation to be added to the case base $\mathcal{CB}$. By Proposition 2, we can write:

$$\Gamma(\sigma_X, \sigma_Y, \mathcal{CB} \cup \{z_{new}\}) = \Gamma(\sigma_X, \sigma_Y, \mathcal{CB}) + \sum_{(z_i, z_j, z_k) \in U(y)} \mathbb{I}_{\text{inc}}(z_i, z_j, z_k)$$

where $U(y)$ is the set of triplets in which z_{new} appears.

Now, the quantity $\Gamma(\sigma_X, \sigma_Y, \mathcal{CB})$ depends only on the existing case base $\mathcal{CB}$ and does not depend on the choice of y. Hence, for any function f such that $f(y) = (C + g(y))$ with C constant, we can write:

$$\arg\min f(y) = \arg\min(C + g(y)) = \arg\min g(y)$$

Finally, since $\Gamma(\sigma_X, \sigma_Y, \mathcal{CB})$ is constant:

$$\begin{aligned}\arg\min_{y \in \mathcal{Y}} \Gamma(\sigma_X, \sigma_Y, \mathcal{CB} \cup \{(x_{\text{new}}, y)\}) &= \arg\min_{y \in \mathcal{Y}} \Big(\Gamma(\sigma_X, \sigma_Y, \mathcal{CB}) + \sum_{(z_i, z_j, z_k) \in U(y)} \mathbb{I}_{\text{inc}}(z_i, z_j, z_k) \Big) \\ &= \arg\min_{y \in \mathcal{Y}} \sum_{(z_i, z_j, z_k) \in U(y)} \mathbb{I}_{\text{inc}}(z_i, z_j, z_k)\end{aligned}$$

Thus, choosing the y that minimizes $\Gamma(\mathcal{CB} \cup \{(x_{\text{new}}, y)\})$ amounts to minimizing only the contribution of the triplets containing z_{new}.

We therefore conclude:

$$\mathrm{CoAT}(x_{\text{new}}) = \arg\min_{y \in \mathcal{Y}} \sum_{(z_i, z_j, z_k) \in U(y)} \mathbb{I}_{\text{inc}}(z_i, z_j, z_k)$$

□

Lemma 2. *i) For any σ_X and σ_Y:*

$$\Gamma(\sigma_X, \sigma_Y, U_6) = \Gamma(\sigma_X, \sigma_Y, U_7) = 0$$

ii) if $\sigma_X(x_1, x_2)$ reaches its maximum only when $x_1 = x_2$, and no case of $\mathcal{CB}$ has the same input as z_{new}, then:

$$\Gamma(\sigma_X, \sigma_Y, U_5) = 0$$

iii) if $\sigma_Y(y_1, y_2)$ reaches its maximum when $y_1 = y_2$, then:

$$\Gamma(\sigma_X, \sigma_Y, U_4) = 0$$

with U_4, U_5, U_6, U_7 defined in Proposition 1.

Proof. Recall that a similarity inversion occurs if, for a triplet (z_i, z_j, z_k), we have:

$$\sigma_X(x_i, x_j) \geq \sigma_X(x_i, x_k) \quad \text{and} \quad \sigma_Y(y_i, y_j) < \sigma_Y(y_i, y_k)$$

Proof of i):

- In U_6, we have $(z_i, z_{\text{new}}, z_{\text{new}})$, so $x_j = x_k$ and $y_j = y_k$. Then:

$$\sigma_X(x_i, x_j) \geq \sigma_X(x_i, x_k) \quad \text{but} \quad \sigma_Y(y_i, y_j) = \sigma_Y(y_i, y_k) \Rightarrow \text{no inversion.}$$

- In U_7, the three elements of the triplet are equal. We therefore have:

$$\sigma_X(x_i, x_j) = \sigma_X(x_i, x_k), \quad \sigma_Y(y_i, y_j) = \sigma_Y(y_i, y_k) \Rightarrow \text{no inversion.}$$

Hence $\Gamma(\sigma_X, \sigma_Y, U_6) = \Gamma(\sigma_X, \sigma_Y, U_7) = 0$.

Proof of ii): In U_5, the triplets are of the form $(z_{\text{new}}, z_j, z_{\text{new}})$. We therefore have $x_i = x_k = x_{\text{new}}$ (both correspond to z_{new}), so the first condition of an inversion reads:

$$\sigma_X(x_{\text{new}}, x_j) \geq \sigma_X(x_{\text{new}}, x_k) = \sigma_X(x_{\text{new}}, x_{\text{new}})$$

But by hypothesis, $\sigma_X(x_{\text{new}}, x_{\text{new}})$ is the **maximum**, and it is reached only if $x_j = x_{\text{new}}$, which is excluded since no case of $\mathcal{CB}$ has the same input as z_{new}. The condition

$$\sigma_X(x_{\text{new}}, x_j) \geq \sigma_X(x_{\text{new}}, x_{\text{new}})$$

is therefore never satisfied. Hence no inversion is possible, and:

$$\Gamma(\sigma_X, \sigma_Y, U_5) = 0$$

Remark. If $\mathcal{CB}$ contained a case z_j with $x_j = x_{\text{new}}$ and $y_j \neq y_{\text{new}}$, the triplets of U_5 could produce inversions. There are at most n of them, so they do not change the $\mathcal{O}(n^2)$ complexity, but they would have to be counted for an exact prediction. In our experiments (Section 5), $\sigma_X(x_i, x_j) = \frac{1}{1+\|x_i - x_j\|_2}$ is maximal only when $x_i = x_j$, and the points are drawn at random in the plane, so this case does not occur in practice.

Proof of iii): For the triplets $(z_{\text{new}}, z_{\text{new}}, z_k) \in U_4$, we have $y_i = y_j$ and therefore:

$$\sigma_Y(y_i, y_j) = \sigma_Y(y_i, y_i)$$

By hypothesis, $\sigma_Y(y_i, y_i)$ is the **maximum**, so it is impossible that:

$$\sigma_Y(y_i, y_j) < \sigma_Y(y_i, y_k)$$

since the left-hand side is maximal. Hence no inversion is possible:

$$\Gamma(\sigma_X, \sigma_Y, U_4) = 0$$

□

Proposition 3. *The complexity of the optimized algorithm based on the set U is $\mathcal{O}(n^2)$.*

Proof. Recall that U is defined as follows in Proposition 1:

$$U = U_1 \sqcup U_2 \sqcup U_3 \sqcup U_4 \sqcup U_5 \sqcup U_6 \sqcup U_7$$

- By Lemma 2 i), $\Gamma(\sigma_X, \sigma_Y, U_6) = \Gamma(\sigma_X, \sigma_Y, U_7) = 0$.
  Hence these sets play no role in our algorithm.
- The subsets U_4, U_5 each contain n elements (we loop over a single element of $\mathcal{CB}$), so they are $\mathcal{O}(n)$.
- The sets U_1, U_2, U_3 each contain n^2 elements (we loop over two elements of $\mathcal{CB}$), so they dominate the complexity.

In the end:

$$|U| = \mathcal{O}(n^2) \quad \Rightarrow \quad \text{The complexity of the optimized algorithm based on the set } U \text{ is } \mathcal{O}(n^2).$$

□

3.2 Proposed optimized algorithm

Based on the previous analysis, we propose the pseudo-code below to formalize the optimized version of the complexity computation for the sets U_1, U_2, U_3 defined above.

Algorithm 2: Complexity induced by z_{new} (optimized $\mathcal{O}(n^2)$ version)

Data: Case base $CB = \{(x_i, y_i)\}_{i=1}^{n}$, new situation $z_{\text{new}} = (x_{\text{new}}, y_{\text{new}})$
Result: Total number of similarity inversions caused by z_{new}

```
cmp ← 0                                                    // Inversion counter
foreach (z_i, z_j) ∈ CB × CB , (i, j) ∈ [1, n]^2 do
    // Case 1:  z_new plays the first role in the triplet
    if σ_X(x_new, x_i) ≥ σ_X(x_new, x_j) and σ_Y(y_new, y_i) < σ_Y(y_new, y_j) then
        cmp ← cmp + 1
    // Case 2:  z_new plays the second role in the triplet
    if σ_X(x_i, x_new) ≥ σ_X(x_i, x_j) and σ_Y(y_i, y_new) < σ_Y(y_i, y_j) then
        cmp ← cmp + 1
    // Case 3:  z_new plays the third role in the triplet
    if σ_X(x_i, x_j) ≥ σ_X(x_i, x_new) and σ_Y(y_i, y_j) < σ_Y(y_i, y_new) then
        cmp ← cmp + 1
return cmp
```

The following algorithm implements CoAT and returns the prediction associated with a new input.

Algorithm 3: Prediction of the class of x_{new} by complexity minimization

Data: Case base $CB = \{(x_i, y_i)\}_{i=1}^{n}$, new input x_{new}, set of possible classes $Y = \{y_1, \ldots, y_k\}$
Result: Predicted class for x_{new}

```
results_list ← empty list
foreach y ∈ Y do
    z_new ← (x_new, y)                                     // Hypothetical new case
    res ← compute_complexity_optimized(CB, z_new)                  // Algorithm 2
    Append res to results_list
index_min ← index of the smallest element of results_list
return Y[index_min]
```

4 Improvement through parallelization

The CoAT algorithm, even optimized to $\mathcal{O}(n^2)$, can become too slow when working with large case bases. In native Python, execution relies solely on the processor (CPU), and explicit loops, even optimized ones, lead to a significant slowdown as soon as the size n increases.

To overcome this constraint, algorithmic improvement through parallelization [5] stands out as an effective solution. It consists in distributing computations over several processing units in order to exploit the parallel computing power of modern architectures, in particular GPUs (Graphics Processing Units). These processors, originally designed for graphics rendering, are now widely used in scientific computing and machine learning, as they provide thousands of cores capable of processing many operations simultaneously.

To make it easier to exploit these powerful architectures, specialized libraries such as PyTorch [4] or CUDA [6] provide tools that allow developers to parallelize their computations more easily. In particular, PyTorch offers an intuitive interface for coding parallelized models and algorithms, with native support for GPU computing.

Our goal is therefore to reformulate the CoAT algorithm as a vectorized version, replacing loops with PyTorch operations on GPU, while preserving the logic of the complexity computation.

4.1 Introduction to PyTorch

PyTorch is based on the same principle as NumPy, but with the ability to run operations either on CPU or on GPU via CUDA (NVIDIA), using the following line of code:

```
import torch
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

This library handles tensors, a data structure similar to an array (or matrix), but generalized to n dimensions. Just like NumPy arrays, tensors can be initialized and operated on directly, which avoids the need for loops while allowing operations to be moved to the GPU as follows:

```
vector = torch.arange(5) // tensor containing the numbers from 0 to 5
vector = vector.to(device) // transfer of the tensor to the GPU
vector_tmp = vector * 2     // -> each element of the tensor * 2   -> returns a
    vector of the same size
```

Moreover, operations can be performed between tensors that do not have the same size or dimension, as long as they are on the same device. For example:

```
vector_tmp = vector + torch.tensor([1, 2, 3, 4, 5]).to(device)
```

The dim parameter makes it possible to apply operations (such as sum, mean, max, etc.) along a specific axis of the tensor, which is essential for correctly handling complex data structures. The following code illustrates this feature:

```
x = torch.tensor([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0]
])

print(torch.sum(x, dim=0))  # column-wise sum
# tensor([5., 7., 9.]) => [1+4, 2+5, 3+6]

print(torch.sum(x, dim=1))  # row-wise sum
# tensor([6., 15.]) => [1+2+3, 4+5+6]
```

Thanks to these mechanisms, PyTorch makes it possible to write vectorized, readable and optimized code, without explicit loops, which is essential for processing large amounts of data.

4.2 From loop-based to parallel computation

Implementing CoAT with PyTorch makes it possible to reduce the loops of the classical implementations, by changing the data types of the function parameters, in particular for the function that counts the number of incompatibilities. Table 1 shows a translation of the code used to implement the classifier.

Let $X \in \mathbb{R}^{n \times d}$ be an array representing the n examples of the case base, each row x_i being an input of dimension d.

We denote:

- `sim_X[i,j]` $= \sigma_X(x_i, x_j)$: a similarity measure between inputs;
- `sim_Y[i,j]` $= \sigma_Y(y_i, y_j)$: a similarity measure between outputs (or labels);
- `triplets`: an array of indices of shape $(m, 3)$ containing m triplets (i, j, k) to evaluate.

Native Python loop	**Vectorized version with PyTorch**

`for i in range(n):`
`  for j in range(n):`
`  dist[i][j] = X[i] - X[j]`
	`X_torch = torch.tensor(X)`
	`dist = torch.cdist(X_torch, X_torch)`

`gamma = 0`
`for i, j, k in triplets:`
`  if sim_X[i,j] >= sim_X[i,k] and \`
`  sim_Y[i,j] < sim_Y[i,k]:`
`  gamma += 1`
	`cond_X = sim_X[:,:,0] >= sim_X[:,:,1]`
	`cond_Y = sim_Y[:,:,0] < sim_Y[:,:,1]`
	`gamma = (cond_X & cond_Y).sum()`

Table 1: Comparison between classical Python implementations and their vectorized PyTorch equivalents.

5 Experiments

The goal of this section is to verify concretely, through tests, the analyses carried out in the previous sections. For this purpose, we use an *NVIDIA GeForce RTX 3050* GPU with 2560 CUDA cores (according to [13]), which we use to test the PyTorch version. The Python versions, on the other hand, only use the CPU.

Chosen application: binary classification of points in the plane

For this project, we chose a simple application of the CoAT method, allowing intuitive visualization and direct control over the data.

Each situation $z_i = (x_i, y_i)$ is a point in the plane $\mathbb{R}^2$:

- the input $x_i \in \mathbb{R}^2$ is represented by the coordinates of the point,
- the output $y_i \in \{\text{red}, \text{blue}\}$ is a **class** label (color).

The similarity functions used are defined as follows:

- **Similarity on inputs**:

$$\sigma_X(x_i, x_j) = \frac{1}{1 + \|x_i - x_j\|_2}$$

  where $\|x_i - x_j\|_2$ denotes the Euclidean distance between x_i and x_j in $\mathbb{R}^2$. This function is maximal for close points and tends to 0 as the distance increases.

- **Similarity on outputs**:

$$\sigma_Y(y_i, y_j) = \begin{cases} 1 & \text{if } y_i = y_j \\ 0 & \text{otherwise} \end{cases}$$

  This is a Boolean function indicating whether two points have the same color.

In this setting, the CoAT classifier, given a new point $x_{\text{new}} \in \mathbb{R}^2$, predicts the color $y \in \{\text{red}, \text{blue}\}$ that minimizes the number of **similarity inversions** (i.e. incompatible triplets) in the case base extended with (x_{new}, y).

This application makes it possible to evaluate the performance of CoAT on a **geometric binary classification** problem, while making the visual analysis of predictions easier.

Experimental protocol

We describe here the protocol chosen to carry out the tests in this project.

Our protocol is as follows:

- The case base is generated as follows: the red points are generated at random within a circle, separately from the blue points.
- We generate a set containing random coordinates in the plane for the points to be tested (without color).
- We run a series of tests for each implementation, varying the number of points in the case base (CB), with fixed similarity measures (described in the previous section).
- We run these tests on several points and average the times obtained, which we display in a plot to obtain a visual representation of our results.

5.1 Implementation

This part describes the structure of our code.

5.1.1 Functions shared by the native Python versions

Some functions are shared by both versions, as they correspond to the fundamental definitions of the model:

- `similarityX(x1, x2)`: measures the similarity between two inputs x_1 and x_2, as defined in the previous section.
- `similarityY(y1, y2)`: measures the similarity between two outputs y_1 and y_2.
- `coat_predict(x_new, CB, ys_unique)` / `coat_predict_optimized(x_new, CB, ys_unique)`: in both cases, the function predicts the best value of y for a new point x_{new} by testing each possible class and selecting the one that minimizes the complexity.

The data are represented as a list of pairs (x_i, y_i), each pair forming one observation of the dataset.

5.1.2 Naive Python version

The naive Python implementation of CoAT relies on 5 main functions:

- The functions described in the previous section.
- `indexes_triplet(data)`: returns all possible index triplets (i,j,k) in data. This function uses yield instead of return in order to create a generator that produces all possible combinations of triplets (i, j, k) on the fly. This choice avoids storing all coordinates in memory, which makes the function lighter and more efficient, especially when the number of combinations becomes large.
- `compute_complexity(data)`: computes the complexity by counting similarity inversions over all possible triplets.

The data are represented as lists of tuples (x_i, y_i) for the classical Python version.

5.1.3 Optimized Python version

The optimized Python implementation of CoAT relies on 6 functions:

- The functions shared with the previous version.
- `indexes_couple(data)`: returns all possible index pairs (i,j) in data. This function again uses yield.
- `evaluate_triplet_inversion(data1,data2,data3)`: helper function that simplifies the computation of similarity inversions.
- `compute_complexity_optimized(data,z_new))`: computes the complexity by counting similarity inversions, going only through the cases in which z_{new} appears.

5.1.4 PyTorch version

For the PyTorch implementation, we used an optimized Euclidean distance function between all pairs (x_i, x_j), based on the formula:

$$\|x_i - x_j\|^2 = \|x_i\|^2 + \|x_j\|^2 - 2\langle x_i, x_j \rangle$$

Our PyTorch version thus relies on 5 main functions:

- `optimized_euclidean_distance(A, B)`: returns the Euclidean distance, optimized for PyTorch.
- `sim_T_CB_1(T_X: torch.Tensor, CB_X: torch.Tensor) -> torch.Tensor`: computes the squared Euclidean distance between each point of T and each point of CB, using the optimized function.
- `sim_T_CB_2(T_Y: torch.Tensor, CB_Y: torch.Tensor) -> torch.Tensor`: computes the color similarity between the points of T and CB.
- `compute_incompatibility(sim_T_CB_1: torch.Tensor, sim_T_CB_2: torch.Tensor) -> torch.Tensor)`: computes the incompatibility for each point of the tensor T.
- `predict_CoAT(T: torch.Tensor, CB: torch.Tensor, Q: list) -> torch.Tensor)`: predicts the color of each point of T according to the CoAT classifier.

5.2 Results and experiments

5.2.1 Illustration of the prediction

The two figures below show the effect of prediction by the CoAT algorithm on a set of points.

On the left, the black points represent elements whose label is unknown. On the right, the same points have been predicted and colored according to the assigned class. This shows the direct impact of the prediction on the configuration of the plane.

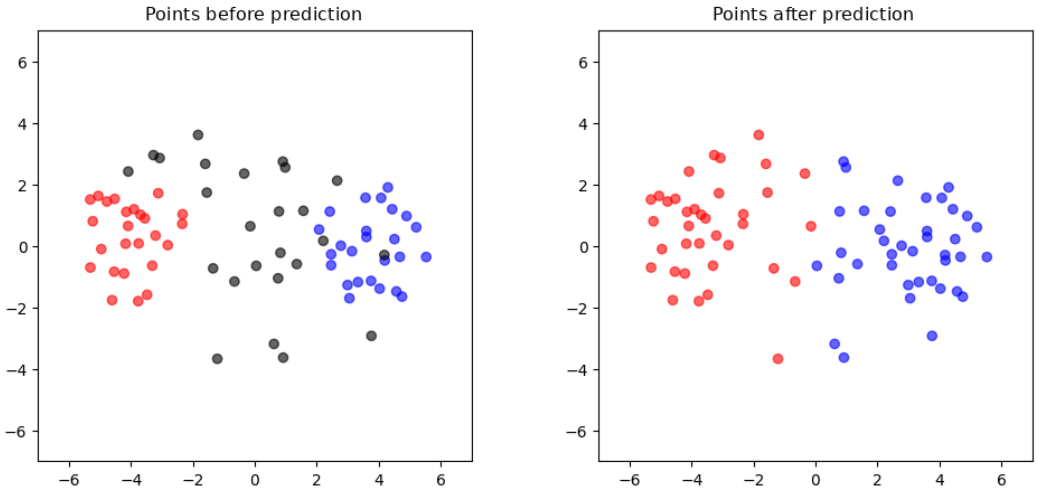


Figure 1: Prediction performed by CoAT on a set of points

5.2.2 Comparison of the Python versions

In this part, we analyze the computation times of our 2 Python versions on case bases of different sizes. Figure 2 illustrates this below.

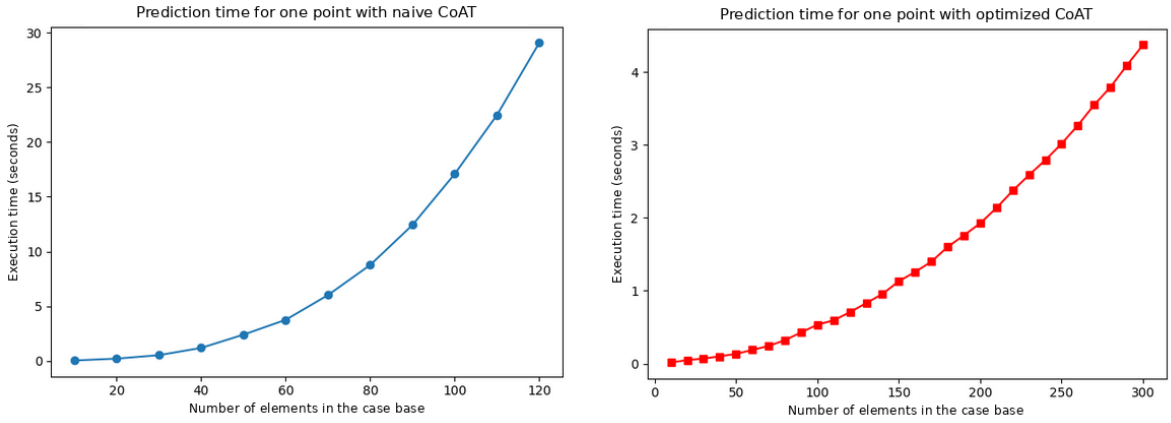


Figure 2: Execution time of the two versions as a function of the size of CB

Naive version For this version, the computation time quickly becomes very large: about 15 seconds to predict one point with a case base of size 100. This is explained by the $\mathcal{O}(n^3)$ complexity of the naive version. This method becomes inefficient on large case bases and when one wants to predict a substantial number of points, with computation times that can reach tens of minutes, or even hours.

Optimized version For this version, the computation time to predict one point with a case base of 100 points is about 0.5 seconds, which represents an improvement by a factor of 30. This version therefore already offers a real advantage over the naive version. It makes it possible to study larger cases with execution times that remain reasonable. However, on larger case bases, for instance a case base of 300 points, the prediction time rises to 4.5 seconds. This result remains acceptable for simple tasks, but becomes problematic in large-scale settings, where computation times can reach tens of minutes.

5.2.3 PyTorch version

Tests on different case bases

First, as in the previous cases, we examine how the computation time evolves as the size of the case base varies. With PyTorch, using the GPU, which allows more computations than the CPU, we were able to run tests at larger scales.

The plot below shows how the computation time needed to predict 100 points evolves as a function of the size of the case base, up to a maximum of 300 elements.

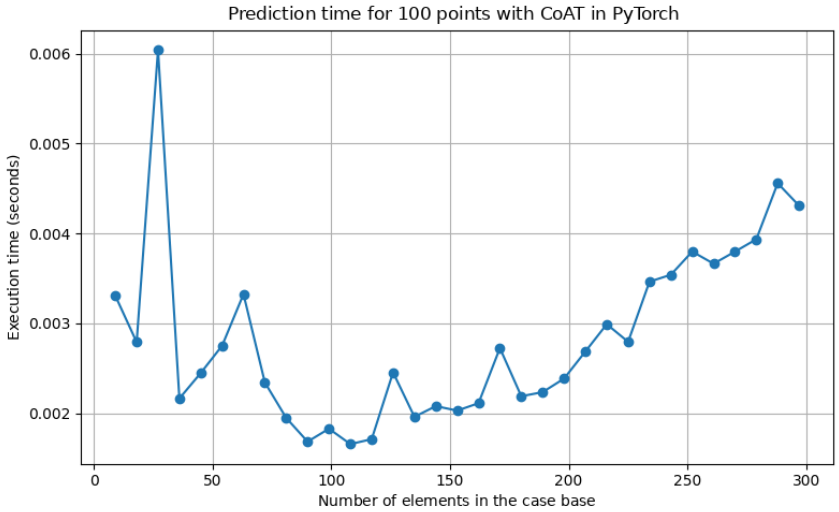


Figure 3: Execution time of the PyTorch version as a function of the size of CB

Compared with a naive implementation, which requires about 15 seconds to predict a single point from a case base of 100 elements according to Figure 2, the PyTorch version predicts 100 points in about 0.004 seconds for a case base of the same size. This improvement corresponds to a performance gain by a factor of 3750, while increasing the number of predicted points by a factor of 100.

For the optimized Python version of CoAT, the computation time is about 0.5 seconds to predict 1 point with a case base of 100 elements, according to Figure 2. Although it remains faster than the naive implementation, this version is still about 125 times slower than the vectorized PyTorch implementation.

These results highlight the efficiency of vectorized processing and parallel execution on GPU in the optimized approach.

However, we cannot infer any increase in computation time for the PyTorch version from these

results.

Tests when both the case base and the number of points to predict vary

In this part, we examine what happens when the size of the case base increases, but also when the number of points to predict increases. Indeed, the computing capacity offered by the vectorized PyTorch version allows us to consider tests on substantially larger case bases.

Figure 4 shows how the computation time evolves when the size of the case base and the number of points to predict increase.

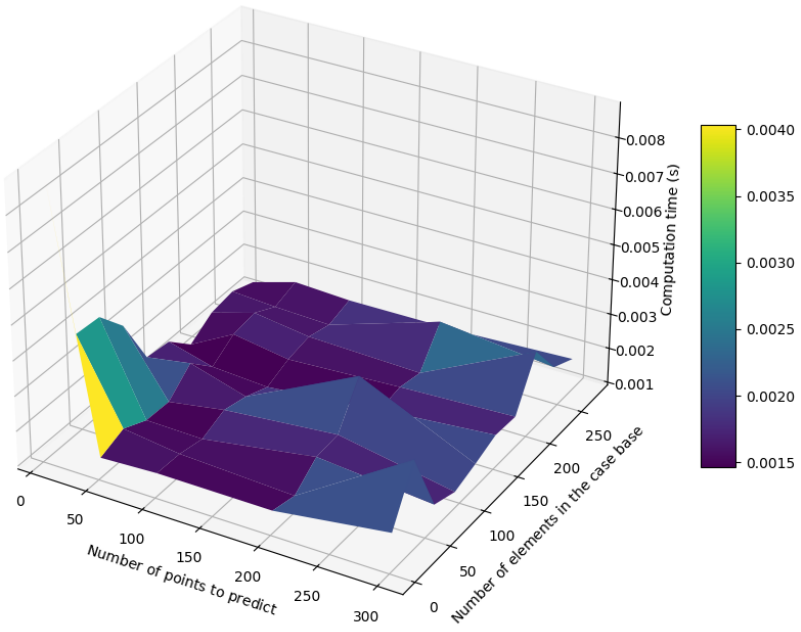


Figure 4: Execution time of the PyTorch version as a function of the size of CB and of the number of points to predict

Despite the joint increase in the size of the case bases and in the number of points to predict, the computation time remains almost constant in the PyTorch version, unlike the naive Python implementation, where significant growth is observed. This behavior is not only expected, but also characteristic of parallel architectures, in particular GPUs. Several phenomena explain this result:

- **Massive GPU parallelism**: GPUs are designed to execute a large number of operations simultaneously. As long as the data size remains within the limits of the available memory and some compute units remain idle, adding more points only causes a marginal additional load, spread over the available cores.
- **Saturation effect (or plateau effect)**: For moderate data sizes, the GPU is often underused. In this phase, increasing the number of points or the size of the case base causes little or no variation in computation time. Only beyond a certain threshold (when memory is saturated or all compute units are busy) does the execution time increase markedly.
- **Vector optimization and matrix computation**: The fundamental operations used in the algorithm (dot products, normalizations, matrix comparisons, etc.) are implemented

in a vectorized and optimized way on GPU. The cost of these operations does not grow linearly with the number of elements as it does in sequential implementations.

Thus, the observed stability of the computation time is not an anomaly, but a direct consequence of the massively parallel nature of the GPU execution environment.

6 Limitations and possible improvements

6.1 Limitations of parallel computing with PyTorch

Although the vectorized and parallelized approach with PyTorch delivers remarkable performance, it is not without limitations. One of the main constraints observed in this setting is related to **GPU memory saturation**. Indeed, when the size of the case base becomes too large, the matrices involved become too large for the GPU, causing a sharp slowdown of the computations or even a failed execution ("out of memory").

This phenomenon generally appears after a **step effect**: as long as the data fit entirely in GPU memory, the computation time remains almost constant. However, beyond a certain critical threshold, the cost increases abruptly. This corresponds to the point where parallelization is no longer enough to compensate for the overhead caused by the size of the matrices.

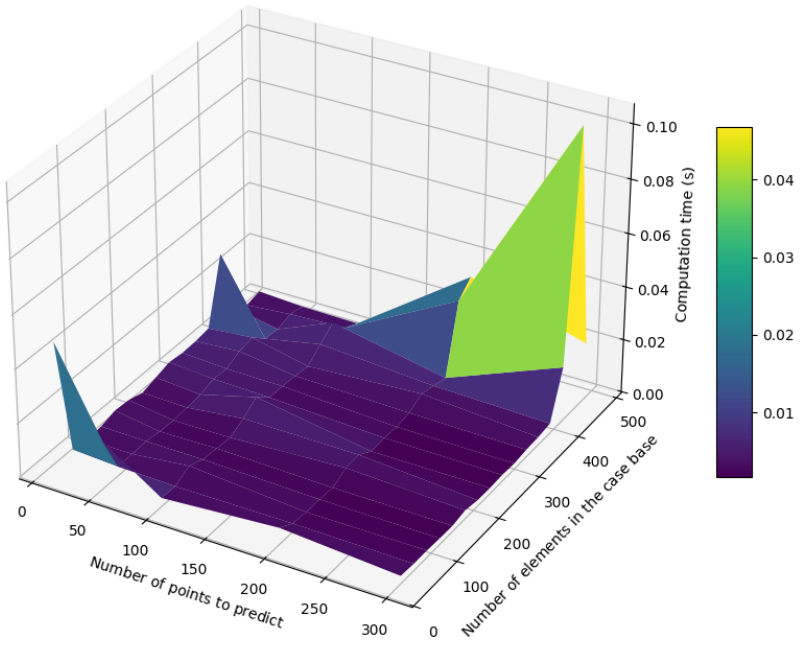


Figure 5: Execution time of the PyTorch version as a function of the size of CB and of the number of points to predict, with GPU saturation

Figure 5 shows the breaking point where the approach stops being efficient, highlighting the intrinsic limitations of parallel computing even in high-performance GPU environments. Here, the computation time rises to 0.1 seconds for a case base of 500 points when predicting 300 points, i.e. a 25-fold increase in computation time compared with the execution time when the GPU is not overloaded.

6.2 Possible improvements

To work around the GPU memory limitation in the PyTorch approach, one improvement strategy we considered consists in dynamically splitting the tensors into smaller sub-blocks. The idea is to process the data in segments compatible with the memory capacity of the GPU.

More precisely, we choose an integer n suited to the hardware configuration (that is, to the memory available on the GPU), then split the matrices or tensors involved into n equal or nearly equal parts. Each block is then processed sequentially on the GPU. This method keeps memory usage below the critical saturation threshold, while preserving the benefits of parallel computing for each sub-block.

This approach introduces a multiplicative factor of about n on the computation time, since the blocks are processed one after the other. However, this increase remains largely acceptable in practice, because vectorized PyTorch operations on GPU are very fast and efficient even when repeated several times.

This trade-off makes it possible to maintain stable and fast execution on large case bases, while avoiding `out of memory` errors.

7 Conclusion

Project summary

This project allowed us to explore in depth the CoAT (Complexity-based Analogical Transfer) classification method, based on minimizing incompatibilities between input and output similarities. After implementing a naive version with cubic complexity ($\mathcal{O}(n^3)$) presented in [1], we proposed an optimized version reducing this complexity to $\mathcal{O}(n^2)$ through an analysis of the relevant triplets. Finally, we implemented a parallelized version using the PyTorch library, taking advantage of vectorized GPU computation to achieve substantially higher performance.

What we learned

This project allowed us to use and deepen both theoretical and practical skills:

- In algorithmics, by identifying redundancies in the triplets and mathematically proving the complexity improvement.
- In implementation, by translating mathematical principles into Python code, then into a vectorized version with PyTorch.
- In experimental analysis, by building a rigorous testing protocol and measuring the impact of the optimizations on computation times.
- In resource management, especially GPU memory, by observing saturation effects and designing solutions to work around them.

Future work

Several directions could be explored following this work:

- Dynamic block management in the PyTorch version to automatically adapt to the available memory.
- Introducing similarity metric learning to adapt σ_X and σ_Y to the data.

- Comparison with other classifiers (such as K-NN or SVM, as in [1]) on real or noisy datasets.

In conclusion, this project was an excellent opportunity to combine mathematical modeling, high-performance programming and experimental analysis, on a real problem arising from machine learning.

References

[1] Fadi Badra, *A Dataset Complexity Measure for Analogical Transfer*, International Joint Conference on Artificial Intelligence (IJCAI), 2020.

[2] Janet L. Kolodner, *An introduction to case-based reasoning*, Artificial Intelligence Review vol 6, 1992.

[3] Todd R. Davis and Stuart J. Russell, *A Logical Approach to Reasoning by Analogy*, IJCAI Int. Jt. Conf. Artif. Intell, 1987.

[4] Adam Paszke *et al.*, *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Advances in Neural Information Processing Systems (NeurIPS), 32, 2019.

[5] J. D. Owens, M. Houston, D. Luebke, S. Green, J. E. Stone, and J. C. Phillips, *GPU computing*, Proceedings of the IEEE, vol. 96, no. 5, pp. 879-899, 2008.

[6] J. Nickolls, I. Buck, M. Garland, and K. Skadron, *Scalable parallel programming with CUDA*, Queue, vol. 6, no. 2, pp. 40-53, 2008.

[7] PyTorch Developers, *CUDA semantics, PyTorch documentation*, `https://pytorch.org/docs/stable/notes/cuda.html`, 2023.

[8] PyTorch Team, *PyTorch Profiler Tutorial*, `https://pytorch.org/tutorials/intermediate/profiler_tutorial.html`, 2023.

[9] Meta AI Research, *Performance Tuning Guide for PyTorch*, `https://pytorch.org/tutorials/recipes/recipes/tuning_guide.html`, 2023.

[10] NVIDIA Corporation, *CUDA C Programming Guide*, `https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html`, 2023.

[11] NVIDIA Corporation, *cuBLAS Library User Guide*, `https://docs.nvidia.com/cuda/cublas/index.html`, 2023.

[12] NVIDIA Developer Tools, *CUDA Occupancy Calculator Documentation*, `https://docs.nvidia.com/cuda/occupancy-calculator/`, 2022.

[13] NVIDIA, *NVIDIA GeForce RTX 3050 Specifications*, `https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/rtx-3050/`, 2022.

Source code

The source code of the project is available on GitHub at the following address:
`https://github.com/AlexandreLB-git/Projet-CoAT`